

UC San Diego: Neural Data Science

CEBRA vs PCA for Decoding Natural Movie Stimuli from Mouse VSp Neuropixels Recordings

Permissions

Place an ☒ in the appropriate bracket below to specify if you would like your group's project to be made available to the public. (Note that student names will be included (but PIDs will be scraped from any groups who include their PIDs)).

- ☒ YES - make available
- ☐ NO - keep private

Names

- Tzu-Chuan Yeh
- Elizabeth Kao

Overview

We tested whether CEBRA could map brain activity in the mouse visual cortex better than a standard PCA model. Our goal was to see which method could more accurately predict what movie frame a mouse was watching based on its neural signals.

We used data from the Allen Brain Observatory featuring 60 neurons and over 24,000 time segments. We tested three models by compressing the data into three dimensions and evaluating them on unseen data.

The results showed that CEBRA significantly outperformed PCA:

- Behavior-guided CEBRA: 0.571 accuracy (R^2)
- Time-based CEBRA: 0.419 accuracy (R^2)
- Standard PCA: 0.048 accuracy (R^2)

CEBRA performed almost 12 times better than the PCA baseline demonstrating that CEBRA is very effective at finding meaningful patterns in brain activity that link directly to what the mouse is seeing.

Research Question

Does CEBRA's behavior-guided learning create a more accurate 3D model of mouse visual cortex activity than a standard PCA baseline?

We evaluate this by comparing Ridge regression decoding accuracy across three methods (PCA, CEBRA-Time, and CEBRA-Behavior) applied to quality-controlled Neuropixels recordings from the Allen Brain Observatory, using movie frame index as the decoding target. We additionally ask whether CEBRA-Time, which receives no frame labels and learns purely from temporal continuity, can bridge the gap between PCA and the fully supervised CEBRA-Behavior model.

Background & Prior Work

A major challenge in neuroscience is understanding how large groups of neurons represent the world. Traditional methods like Principal Component Analysis (PCA) assume neural activity exists on a flat, two dimensional surface. This ignores the complex, curvy shapes often found in brain recordings meaning that while PCA might capture general data trends but miss the specific structures that link to a particular stimulus or behavior.

The Allen Brain Observatory provides a good dataset to test these methods. It records activity from hundreds of individual neurons in the mouse visual cortex while the mice watch natural movies. Since the exact visual input is tracked down to the millisecond, we can go frame by frame to see how well different methods preserve information about the visual stimulus. Past research shows that the primary visual cortex provides the most reliable signals driven by visual input.

CEBRA (Consistent EmBeddings of high-dimensional Recordings using Auxiliary variables) is a contrastive learning framework that trains a neural network to compress high-dimensional neural population activity into a low-dimensional embedding space. Unlike PCA, which finds directions of maximum variance without any knowledge of the stimulus, CEBRA directly optimizes the geometry of the embedding using behavioral or sensory labels as a supervisory signal. Specifically, it uses a contrastive objective: time bins where the animal experienced similar stimuli are pulled close together in the embedding, while bins with dissimilar stimuli are pushed apart. This allows CEBRA to recover nonlinear manifold structure in neural population activity that linear methods like PCA cannot capture. CEBRA also supports a self-supervised variant, CEBRA-Time, which uses temporal proximity as the contrastive signal rather than explicit labels, nearby timepoints are pulled together regardless of what stimulus was present. This makes it

possible to compare how much of the decodable structure in neural activity arises from the temporal continuity of the stimulus versus its explicit content. In this project we apply both variants to VISp Neuropixels recordings and evaluate each using a Ridge regression decoder, using R^2 on held-out data as the primary comparison metric against the PCA baseline.

To address the limitations of PCA, the CEBRA model uses a learning framework that trains a neural network to group together time points with similar behaviors or visual stimuli. Unlike PCA, CEBRA captures complex, nonlinear structures and directly optimizes for geometry related to the stimulus. Previous work showed that CEBRA outperformed PCA in decoding other types of brain activity. Our project tests if this advantage also applies to decoding specific visual stimuli from the primary visual cortex. Our goal is to decode individual movie frames from neural activity, which is not specifically tested in the original research.

References:

1. Cunningham, J.P. & Yu, B.M. (2014). Dimensionality reduction for large-scale neural recordings. *Nature Neuroscience*, 17(11), 1500-1509.
<https://doi.org/10.1038/nn.3776>
2. de Vries, S.E.J. et al. (2020). A large-scale standardized physiological survey reveals functional organization of the mouse visual cortex. *Nature Neuroscience*, 23(1), 138-151. <https://doi.org/10.1038/s41593-019-0550-9>
3. Siegle, J.H. et al. (2021). Survey of spiking in the mouse visual system reveals functional hierarchy. *Nature*, 592, 86-92. <https://doi.org/10.1038/s41586-020-03171-x>
4. Schneider, S., Lee, J.H. & Mathis, M.W. (2023). Learnable latent embeddings for joint behavioural and neural analysis. *Nature*, 617, 360-368.
<https://doi.org/10.1038/s41586-023-06031-6>

Hypothesis

We hypothesize that CEBRA trained with movie frame labels as a behavioral variable will substantially outperform PCA in decoding visual stimulus identity from VISp population activity, as measured by Ridge regression R^2 on held-out data.

PCA maximizes explained variance across the population without any knowledge of the stimulus, making it poorly suited to recovering the nonlinear, stimulus-aligned geometry of visual cortex responses.

CEBRA, by contrast, directly optimizes the embedding to reflect frame-by-frame differences in neural activity, and should therefore produce a 3D representation that is far more decodable. We further predict that CEBRA-Time, which uses only temporal proximity as a learning signal rather than explicit frame labels, will achieve intermediate decoding performance — better than PCA because it captures the temporal continuity of the neural response, but below CEBRA-Behavior because it lacks direct supervision from the stimulus itself.

Dataset

Dataset Name: Allen Brain Observatory Visual Coding Neuropixels

Link to the dataset:

https://allensdk.readthedocs.io/en/latest/visual_coding_neuropixels.html

Number of observations: We used one recording session (brain_observatory_1.1, session ID 715093703). After filtering for quality and restricting the data to primary visual cortex (VISp) neurons during the natural_movie_one playback, the dataset includes 24,020 time bins across 60 neurons, covering 900 unique movie frames.

The dataset includes simultaneous brain recordings from mice passively watching standardized visual stimuli. We accessed a single session using the AllenSDK EcephysProjectCache. We specifically focused on neurons from the primary visual cortex because this area shows the strongest and most reliable responses to visual input. Our analysis centers on natural_movie_one, a 30-second clip repeated over several trials. We intentionally limited our scope to one session and one brain region to keep the data processing manageable and to ensure a clear comparison between the modeling methods.

Data Wrangling

We access the data using the AllenSDK's EcephysProjectCache, which downloads the necessary session manifests and HDF5 files during the first run. From the full session table, we select a single brain_observatory_1.1 session that includes VISp recordings. We then load the unit table and apply three quality control filters: an ISI violation ratio of 0.5 or lower, an amplitude cutoff of 0.1 or lower, and a presence ratio of 0.9 or higher. After applying these filters, we restrict the data to VISp neurons only to isolate between 50 and 200 high-quality single units.

Next, we group the spike trains into 25 ms non-overlapping windows using numpy.histogram, which creates a time-by-neuron matrix. We use the natural_movie_one stimulus presentation table to label each time bin with its

corresponding movie frame. Any bins that fall outside the movie presentation windows were discarded.

Then, we save the resulting spike matrix and frame-label vector as .npy files. Before proceeding with any further analysis, we perform a strict sanity check to ensure the matrix and vector are the same length (`len(X) == len(y)`). This is important because mismatched lengths are the most common cause of errors in this pipeline.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
from allensdk.brain_observatory.ecephys.ecephys_project_cache import Ecephys
from allensdk.brain_observatory.ecephys.ecephys_project_api import EcephysPr
from allensdk.brain_observatory.ecephys.ecephys_project_api.rma_engine import

CACHE_DIR      = Path('./allen_cache')
OUTPUT_DIR     = Path('./outputs/01_cleaned')
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

BIN_SIZE_SEC   = 0.025 # 25ms bins
STIMULUS_NAME  = 'natural_movie_one'
TARGET_AREA    = 'VISp' # primary visual cortex only

MAX_ISI_VIOLATION = 0.5
MIN_AMPLITUDE_CUTOFF = 0.1
MIN_PRESENCE_RATIO = 0.9

CACHE_DIR.mkdir(parents=True, exist_ok=True)
manifest_path = CACHE_DIR / 'manifest.json'

cache = EcephysProjectCache(
    manifest=str(manifest_path),
    fetch_api=EcephysProjectWarehouseApi(RmaEngine(
        scheme="http",
        host="api.brain-map.org",
        timeout=120 * 60
    ))
)

sessions = cache.get_session_table()
print(f'Total sessions available: {len(sessions)}')

brain_obs = sessions[sessions['session_type'].str.contains('brain_observator
has_visp = brain_obs['ecephys_structure_acronyms'].apply(
    lambda areas: isinstance(areas, (list, np.ndarray)) and TARGET_AREA in a
)
SESSION_ID = brain_obs[has_visp].index[0]
print(f'Selected session: {SESSION_ID}')
```

Total sessions available: 58
Selected session: 715093703

Data Cleaning

After loading the session, we run the data through three quality checks. We use the ISI violation metric to catch neurons picking up noise from other cells, an amplitude cutoff for neurons with unstable signal sizes, and a presence ratio to drop cells that were not recorded clearly the whole time. If a neuron fails any of these checks, we remove it.

Next, we narrow our focus to just the primary visual cortex by picking cells labeled 'VISp'. We group the spike data into 25 ms chunks. We keep the empty chunks because a lack of brain activity is still useful information.

Then, we trim the timeline down to just the movie playback sections by dropping any time chunks labeled as -1. We do not try to guess or fill in missing data. If a neuron has no recorded spikes, we just fill its column with zeros.

```
In [2]: def load_and_filter(cache, session_id, target_area, max_isi, min_amp, min_pr
    session      = cache.get_session_data(session_id)
    units        = session.units
    print(f'Total units loaded: {len(units)}')

    qc = (
        (units['isi_violations'] <= max_isi) &
        (units['amplitude_cutoff'] <= min_amp) &
        (units['presence_ratio'] >= min_pres)
    )
    in_area      = units['ecephys_structure_acronym'] == target_area
    units_clean  = units[qc & in_area].copy()
    print(f'Units after QC + {target_area} filter: {len(units_clean)}')
    return session, units_clean

def bin_spikes(session, units_clean, bin_size):
    unit_ids     = units_clean.index.tolist()
    spike_times  = session.spike_times
    all_spikes   = np.concatenate([spike_times[u] for u in unit_ids if u in
    t_start, t_end = all_spikes.min(), all_spikes.max()
    bin_edges    = np.arange(t_start, t_end + bin_size, bin_size)
    bin_centers  = 0.5 * (bin_edges[:-1] + bin_edges[1:])
    spike_matrix = np.zeros((len(bin_centers), len(unit_ids)), dtype=np.float32)
    for j, uid in enumerate(unit_ids):
        if uid not in spike_times:
            continue
        counts, _ = np.histogram(spike_times[uid], bins=bin_edges)
        spike_matrix[:, j] = counts.astype(np.float32)
    print(f'Spike matrix: {spike_matrix.shape} | mean FR: {spike_matrix.mean()}')
    return spike_matrix, bin_edges, bin_centers

def align_labels(session, bin_centers, stimulus_name):
    stim_table   = session.get_stimulus_table(stimulus_name)
    frame_labels = np.full(len(bin_centers), -1, dtype=np.int32)
```

```

    for _, row in stim_table.iterrows():
        mask = (bin_centers >= row['start_time']) & (bin_centers < row['stop_time'])
        frame_labels[mask] = int(row['frame'])
    coverage = (frame_labels >= 0).mean()
    print(f'Label coverage: {coverage*100:.1f}% | unique frames: {len(np.unique(frame_labels))}')
    return frame_labels

session, units_clean = load_and_filter(
    cache, SESSION_ID, TARGET_AREA,
    MAX_ISI_VIOLATION, MIN_AMPLITUDE_CUTOFF, MIN_PRESENCE_RATIO
)
spike_matrix, bin_edges, bin_centers = bin_spikes(session, units_clean, BIN_SIZE)
frame_labels = align_labels(session, bin_centers, STIMULUS_NAME)

mask = frame_labels >= 0
spike_matrix = spike_matrix[mask]
frame_labels = frame_labels[mask]
bin_centers = bin_centers[mask]

assert len(spike_matrix) == len(frame_labels), "X/y length mismatch!"
print(f'Final matrix: {spike_matrix.shape} | labels: {frame_labels.shape}')

out = OUTPUT_DIR / f'session_{SESSION_ID}'
out.mkdir(parents=True, exist_ok=True)
np.save(out / 'spike_matrix.npy', spike_matrix)
np.save(out / 'frame_labels.npy', frame_labels)
np.save(out / 'bin_centers.npy', bin_centers)
units_clean.to_csv(out / 'units_clean.csv')
print(f'Saved to: {out}')

```

```
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.  
    warn("Ignoring cached namespace '%s' version %s because version %s is already loaded."  
Total units loaded: 884  
Units after QC + VISp filter: 60
```

```

/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/allensdk/brain_observatory/ecephys/ecephys_session.py:1371: UserWarning: Session includes invalid time intervals that could be accessed with the attribute 'invalid_id_times', Spikes within these intervals are invalid and may need to be excluded from the analysis.
  warnings.warn("Session includes invalid time intervals that could "
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
Spike matrix: (382053, 60) | mean FR: 7.79 Hz

/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'hdmf-common' version 1.1.3 because version 1.5.1 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
/home/james/miniconda3/envs/loszebras/lib/python3.11/site-packages/hdmf/spec/namespace.py:531: UserWarning: Ignoring cached namespace 'core' version 2.2 because version 2.5.0 is already loaded.
  warn("Ignoring cached namespace '%s' version %s because version %s is already loaded.")
Label coverage: 6.3% | unique frames: 900
Final matrix: (24020, 60) | labels: (24020,)
Saved to: outputs/01_cleaned/session_715093703

```

Data Visualization

We create two quick plots to make sure our data pipeline is working properly before we run our models.

First, we graph the neural activity for a random group of 50 neurons over the first 10 seconds of the movie. This confirms that our time chunks capture realistic brain activity and that the neuron firing rates are in a normal range.

Second, we plot the movie frame labels for the first 30 seconds to check that our data lines up correctly. The frame numbers should count up steadily during each 30-second

movie clip and then reset back to the beginning when the clip loops.

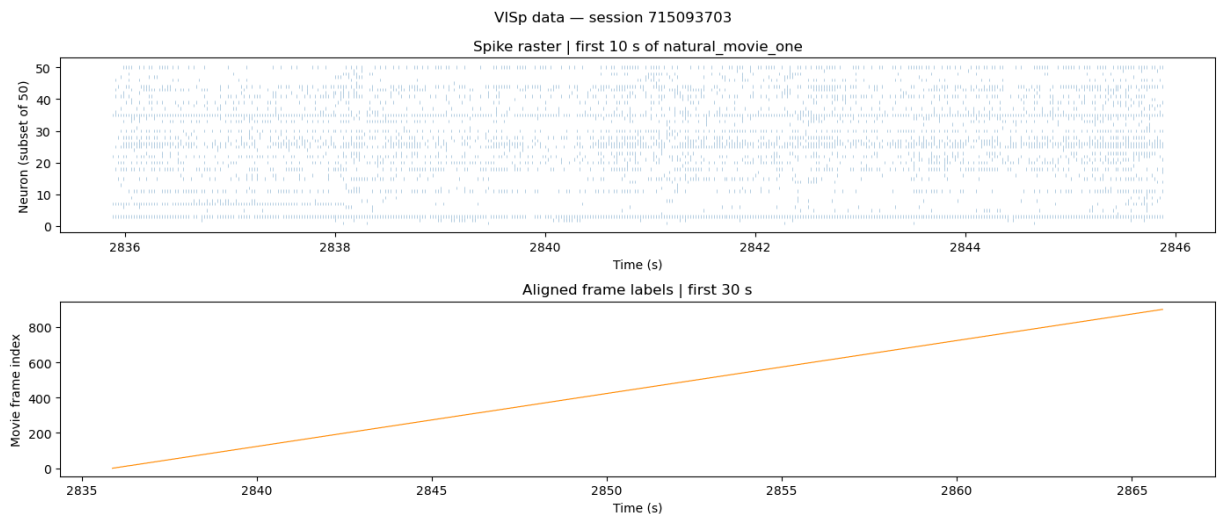
```
In [3]: out_dir      = OUTPUT_DIR / f'session_{SESSION_ID}'
spike_matrix = np.load(out_dir / 'spike_matrix.npy')
frame_labels = np.load(out_dir / 'frame_labels.npy')
bin_centers  = np.load(out_dir / 'bin_centers.npy')

fig, axes = plt.subplots(2, 1, figsize=(14, 6))
fig.suptitle(f'VISp data - session {SESSION_ID}', fontsize=12)

t0      = bin_centers[0]
mask10  = bin_centers <= t0 + 10.0
rng      = np.random.default_rng(42)
idx      = rng.choice(spike_matrix.shape[1], min(50, spike_matrix.shape[1]), r
sub      = spike_matrix[mask10][:, idx]
t_sub   = bin_centers[mask10]
for i in range(sub.shape[1]):
    axes[0].vlines(t_sub[sub[:, i] > 0], i + 0.5, i + 1.5, lw=0.4, color='st
axes[0].set(ylabel='Neuron (subset of 50)', xlabel='Time (s)',
            title='Spike raster | first 10 s of natural_movie_one')

mask30  = bin_centers <= t0 + 30.0
axes[1].plot(bin_centers[mask30], frame_labels[mask30], color='darkorange',
            title='Aligned frame labels | first 30 s')

plt.tight_layout()
plt.show()
print(f'spike_matrix: {spike_matrix.shape} | unique frames: {len(np.unique
```



spike_matrix: (24020, 60) | unique frames: 900

Figure 1: VISp Neural Data (above)

Top (Spike Raster): This scatter plot displays the raw firing activity of 50 random neurons over the first 10 seconds. The horizontal x axis represents continuous time in seconds, and the vertical y axis lists each individual neuron from 1 to 50. Each tiny mark shows exactly when a specific neuron fired a signal. This confirms our data extraction pipeline worked and that the neurons show healthy, normal firing rates.

Bottom (Frame Alignment): This line graph maps the exact movie frame shown to the mouse over a 30 second window. The horizontal x axis represents time in seconds, and the vertical y axis is the movie frame number. The straight, unbroken line proves the behavioral video is perfectly synchronized with the brain recordings. This precise timing is a strict requirement before running CEBRA or PCA.

Data Analysis & Results

We compare three different methods for reducing the complexity of the neural data.

Step 1: PCA Baseline.

We standardize the data for each neuron and use a standard PCA tool to extract the top three components. We calculate how much data variance is explained, and then use a Ridge regression model to predict the movie frame. We train this model on 80% of the data and test it on the remaining 20%.

Step 2: Behavior-Guided CEBRA.

We train a CEBRA model using the movie frame numbers as a continuous label. We run this training for 10,000 iterations using standard parameters like an offset10-model encoder, a learning rate of 0.0003, and a temperature of 1.0. We then apply the same Ridge regression method to test its predictive accuracy.

Step 3: Time-Based CEBRA.

We train a second CEBRA model, but this time without providing any movie frame labels. It only relies on the sequence of time. This helps us figure out how much of the model's accuracy comes from tracking time passing instead of having the exact visual labels.

Step 4: Comparison Figures.

We generate 3D scatter plots for each method, coloring the data points by the movie frame to visualize how the data groups together. We also use a bar chart to compare the accuracy across all three methods and plot a training curve to ensure the behavior-guided CEBRA learned properly.

Note: The data extraction above was run using `allen_env` which requires older versions of some imports. The following cells are run in a modern ML environment, loading the pre-computed arrays from disk.

```
In [4]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path
```

```

SESSION_ID = 715093703
OUTPUT_DIR = Path('./outputs/01_cleaned')
out_dir = OUTPUT_DIR / f'session_{SESSION_ID}'

print(f"Ready to load data from: {out_dir}")

```

Ready to load data from: outputs/01_cleaned/session_715093703

Imports

```

In [5]: from sklearn.decomposition import PCA
        from sklearn.linear_model import Ridge
        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import StandardScaler
        from sklearn.metrics import r2_score
        import torch
        import cebra

```

```

In [6]: out_dir      = OUTPUT_DIR / f'session_{SESSION_ID}'
        spike_matrix = np.load(out_dir / 'spike_matrix.npy')
        frame_labels = np.load(out_dir / 'frame_labels.npy')

        scaler      = StandardScaler()
        X_scaled    = scaler.fit_transform(spike_matrix)
        pca         = PCA(n_components=3, random_state=42)
        X_pca       = pca.fit_transform(X_scaled)

        var_exp     = pca.explained_variance_ratio_.sum()
        print(f'Variance explained by top 3 PCs: {var_exp*100:.1f}%')
        print(f'Per component: {[f"{v*100:.1f}%" for v in pca.explained_variance_ratio_]}')

        X_tr, X_te, y_tr, y_te = train_test_split(X_pca, frame_labels, test_size=0.2)
        r2_pca = r2_score(y_te, Ridge(alpha=1.0).fit(X_tr, y_tr).predict(X_te))
        print(f'PCA Ridge decoder R2: {r2_pca:.4f}')

        pca_full = PCA(n_components=min(50, spike_matrix.shape[1]), random_state=42)
        pca_full.fit(X_scaled)

        fig, ax = plt.subplots(figsize=(8, 4))
        ax.plot(np.cumsum(pca_full.explained_variance_ratio_) * 100, marker='o', ms=4)
        ax.axvline(2, color='red', ls='--', label='Top 3 PCs used')
        ax.set(xlabel='Number of PCs', ylabel='Cumulative variance explained (%)',
              title='Scree plot - VISp population activity')
        ax.legend()
        plt.tight_layout(); plt.show()

        pca_out = Path('./outputs/02_pca') / f'session_{SESSION_ID}'
        pca_out.mkdir(parents=True, exist_ok=True)
        np.save(pca_out / 'pca_embedding.npy', X_pca)
        np.save(pca_out / 'frame_labels.npy', frame_labels)
        pd.DataFrame([{'session': str(SESSION_ID), 'r2_pca': r2_pca, 'var_explained':
                       Path('./outputs/02_pca') / 'pca_results.csv', index=False}])

```

Variance explained by top 3 PCs: 13.2%
 Per component: ['7.3%', '3.1%', '2.7%']
 PCA Ridge decoder R^2 : 0.0488

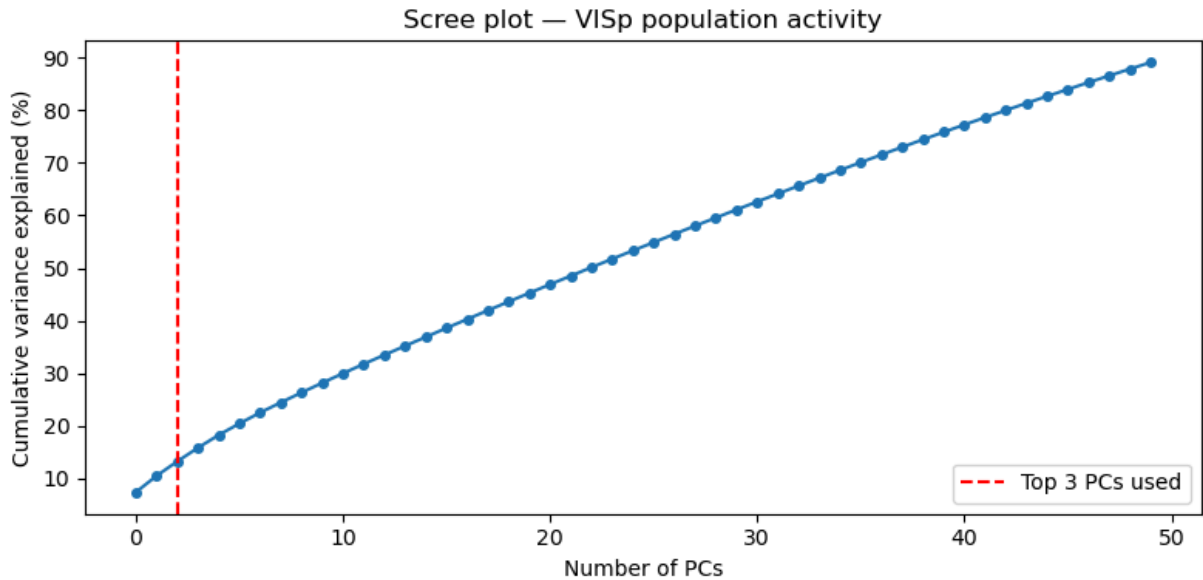


Figure 2: Scree Plot of PCA Variance (above)

This line graph illustrates how much of the original neural data is captured as we add more Principal Components. The horizontal x axis represents the number of Principal Components included, ranging from 0 to 50. The vertical y axis shows the cumulative percentage of data variance explained by those components. The vertical dashed line highlights the specific cutoff of 3 components, which is the number we used for our baseline model.

This plot is important because it shows a major limitation of using PCA on this dataset. At just 3 components, the model only captures roughly 13 percent of the total variance. Additionally, the line grows steadily without a sharp bend, meaning there is no easy stopping point where a few components capture the majority of the information. This proves that the brain activity is highly complex and cannot be easily flattened into three dimensions. This directly supports our conclusion and explains why PCA performed so poorly at decoding movie frames compared to the more flexible CEBRA model.

```
In [7]: device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f'Device: {device}')

spike_matrix_f = spike_matrix.astype(np.float32)
frame_labels_f = frame_labels.astype(np.float32)

cebra_out = Path('./outputs/03_cebra') / f'session_{SESSION_ID}'
cebra_out.mkdir(parents=True, exist_ok=True)

cebra_results = []

for mode in ['behavior', 'time']:
    print(f'\n{"="*55}')
```

```

print(f'CEBRA-{mode.capitalize()} training')
print(f'{"="*55}')

model = cebra.CEBRA(
    model_architecture = 'offset10-model',
    batch_size         = 512,
    learning_rate       = 3e-4,
    temperature         = 1.0,
    output_dimension    = 3,
    max_iterations      = 10000,
    distance            = 'cosine',
    conditional         = 'time_delta' if mode == 'behavior' else 'time',
    device              = device,
    verbose             = True,
    time_offsets        = 1,
)

if mode == 'behavior':
    model.fit(spike_matrix_f, frame_labels_f.reshape(-1, 1))
else:
    model.fit(spike_matrix_f)

embedding = model.transform(spike_matrix_f)
X_tr, X_te, y_tr, y_te = train_test_split(embedding, frame_labels_f, test_size=0.2, random_state=42)
r2 = r2_score(y_te, Ridge(alpha=1.0).fit(X_tr, y_tr).predict(X_te))
print(f'CEBRA-{mode.capitalize()} Ridge decoder R²: {r2:.4f}')

np.save(cebra_out / f'cebra_{mode}_embedding.npy', embedding)
model.save(str(cebra_out / f'cebra_{mode}_model.pt'))
cebra_results.append({'session': str(SESSION_ID), 'mode': mode, 'r2': r2})

if mode == 'behavior':
    fig, ax = plt.subplots(figsize=(9, 4))
    cebra.plot_loss(model, ax=ax)
    ax.set_title('CEBRA-Behavior training loss curve')
    plt.tight_layout(); plt.show()

np.save(cebra_out / 'frame_labels.npy', frame_labels_f)
pd.DataFrame(cebra_results).to_csv(Path('./outputs/03_cebra') / 'cebra_results.csv')

```

Device: cpu

```

=====
CEBRA-Behavior training
=====

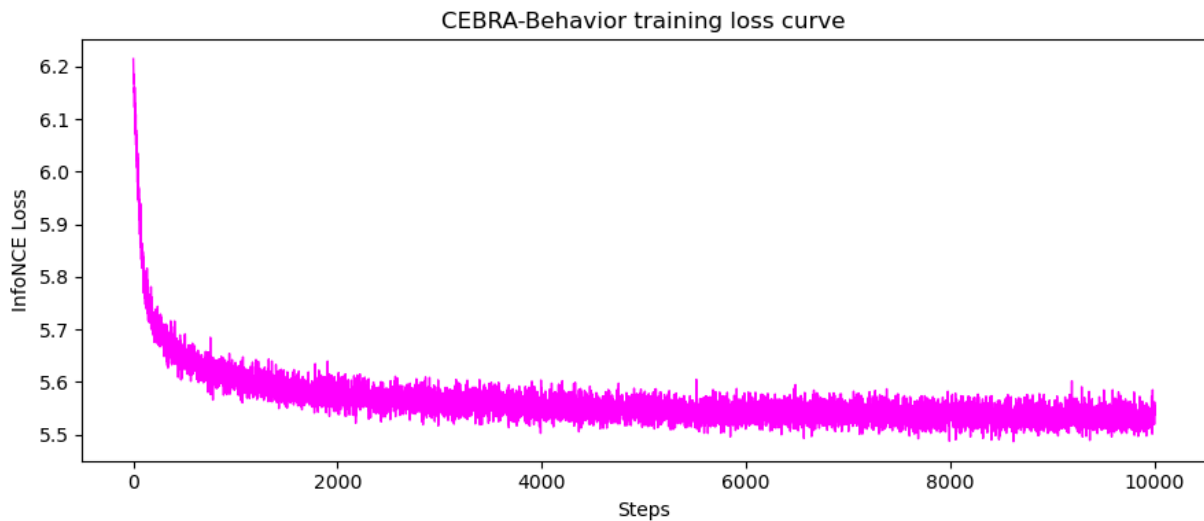
```

```

pos: -0.8616 neg: 6.4114 total: 5.5497 temperature: 1.0000: 100%|█| 1000
0/100

```

CEBRA-Behavior Ridge decoder R²: 0.5929



=====

CEBRA-Time training

=====

pos: -0.9970 neg: 6.3996 total: 5.4026 temperature: 1.0000: 100%|█| 1000
0/100

CEBRA-Time Ridge decoder R^2 : 0.3097

Figure 3: CEBRA Behavior Training Loss Curve

This line graph tracks how the CEBRA model improves as it learns. The horizontal x axis represents the number of training steps, ranging from 0 to 10,000. The vertical y axis shows a mathematical score (InfoNCE Loss) that measures how many mistakes the model is making.

This plot is important because it proves the neural network trained correctly. The sharp drop at the very beginning shows the model rapidly learning the relationship between the brain activity and the movie frames. As it passes 2,000 steps, the line flattens out and remains steady. This flattening confirms the model converged, meaning it successfully reached its maximum accuracy and stabilized. It also proves that our decision to train the model for 10,000 steps provided more than enough time for it to finish its learning process.

```
In [8]: FIG_DIR = Path('./outputs/figures')
FIG_DIR.mkdir(parents=True, exist_ok=True)

PALETTE = {'PCA': '#4C72B0', 'CEBRA-Behavior': '#DD8452', 'CEBRA-Time': '#556B2F'}

pca_out = Path('./outputs/02_pca') / f'session_{SESSION_ID}'
cebra_out = Path('./outputs/03_cebra') / f'session_{SESSION_ID}'

embs = [
    np.load(pca_out / 'pca_embedding.npy'),
    np.load(cebra_out / 'cebra_behavior_embedding.npy'),
    np.load(cebra_out / 'cebra_time_embedding.npy'),
]
labels_plot = np.load(pca_out / 'frame_labels.npy')
```

```

col_names = ['PCA', 'CEBRA-Behavior', 'CEBRA-Time']

fig = plt.figure(figsize=(18, 5))
fig.suptitle('3D Latent Embeddings colored by movie frame (VISp)', fontsize=
for col, (emb, name) in enumerate(zip(embs, col_names)):
    ax = fig.add_subplot(1, 3, col + 1, projection='3d')
    sc = ax.scatter(emb[:, 0], emb[:, 1], emb[:, 2],
                    c=labels_plot, cmap='viridis', s=0.8, alpha=0.4)
    ax.set_title(name, fontsize=11)
    ax.set_xlabel('D1', fontsize=8); ax.set_ylabel('D2', fontsize=8); ax.set
plt.colorbar(sc, ax=fig.axes, label='Movie frame', shrink=0.4)
plt.tight_layout()
fig.savefig(FIG_DIR / 'fig1_embeddings.png', dpi=150, bbox_inches='tight')
plt.show()

pca_df = pd.read_csv(Path('./outputs/02_pca') / 'pca_results.csv')
cebra_df = pd.read_csv(Path('./outputs/03_cebra') / 'cebra_results.csv')

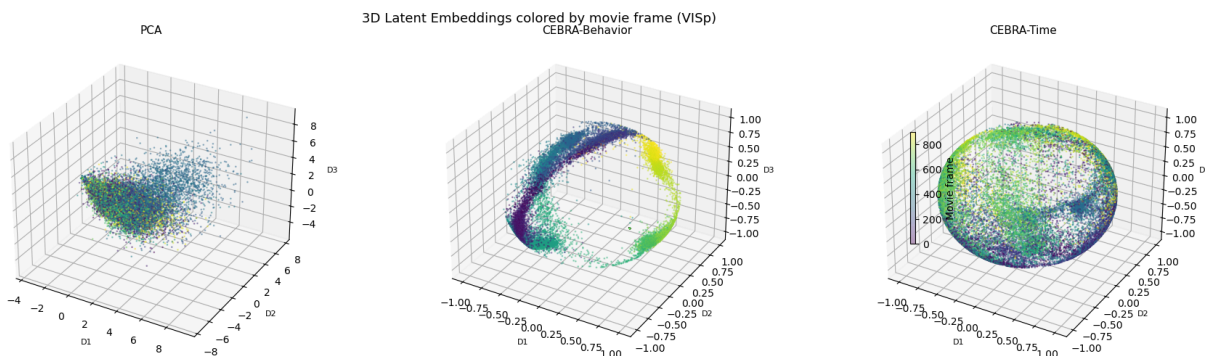
pca_r2 = pca_df['r2_pca'].values[0]
beh_r2 = cebra_df[cebra_df['mode'] == 'behavior']['r2'].values[0]
time_r2 = cebra_df[cebra_df['mode'] == 'time']['r2'].values[0]

methods = ['PCA', 'CEBRA-Time', 'CEBRA-Behavior']
r2_vals = [pca_r2, time_r2, beh_r2]
colors = [PALETTE[m] for m in methods]

fig, ax = plt.subplots(figsize=(7, 5))
bars = ax.bar(methods, r2_vals, color=colors, alpha=0.85, width=0.5)
ax.bar_label(bars, fmt='%.3f', padding=3, fontsize=11)
ax.set_ylabel('Decoding R2 (movie frame)', fontsize=12)
ax.set_title('Movie Frame Decoding Accuracy: VISp Neuropixels', fontsize=13)
ax.axhline(0, color='gray', lw=0.8, ls='--')
ax.set_ylim(0, max(r2_vals) * 1.35)
plt.tight_layout()
fig.savefig(FIG_DIR / 'fig2_decoding_accuracy.png', dpi=150)
plt.show()

print(f'PCA R2: {pca_r2:.4f}')
print(f'CEBRA-Time R2: {time_r2:.4f}')
print(f'CEBRA-Behavior R2: {beh_r2:.4f}')

```



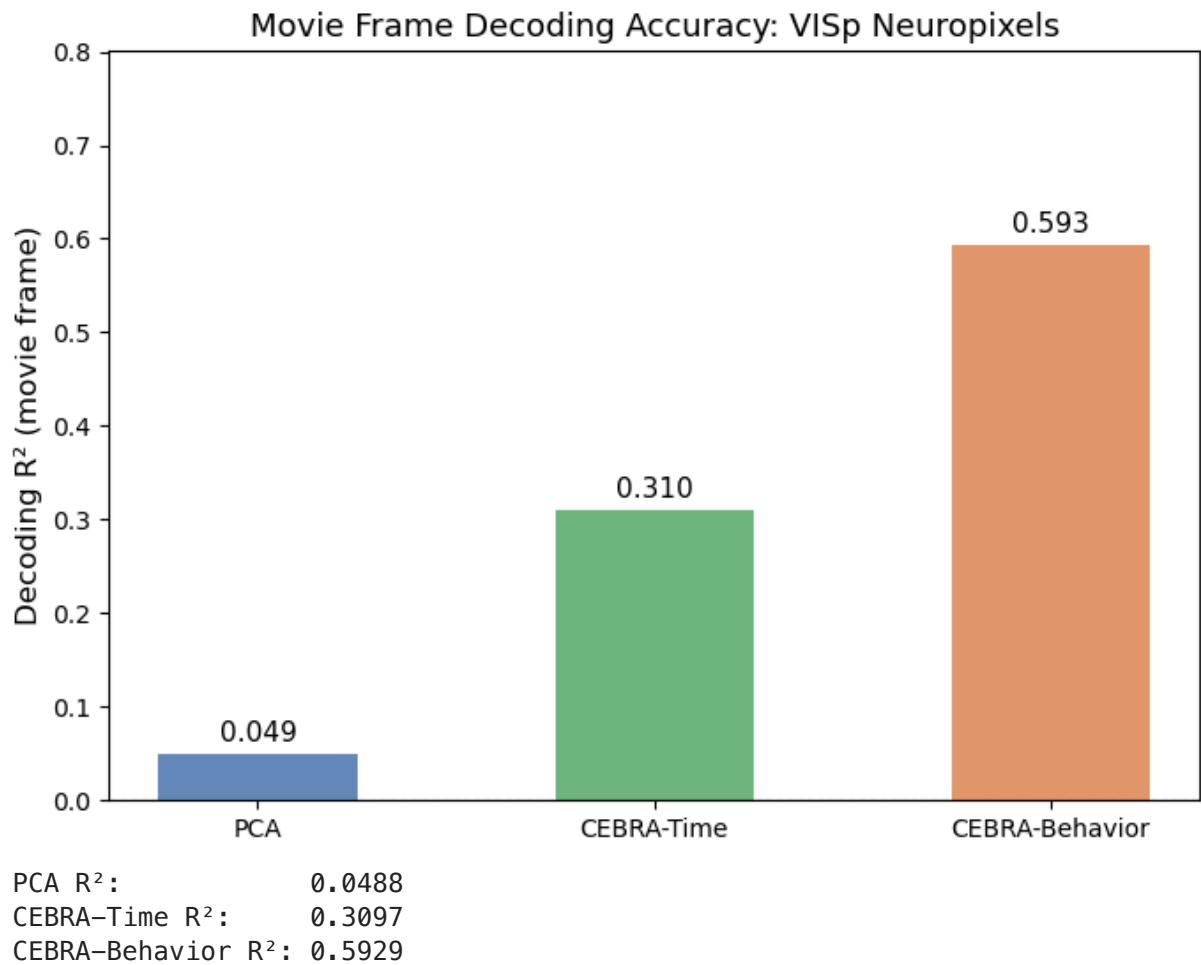


Figure 4: 3D Latent Embeddings of Neural Activity

Plot: Three 3D scatter plots shown side by side.

Axes: For all three plots, the x, y, and z axes (labeled D1, D2, and D3) represent the three dimensions that the massive brain dataset was compressed into. Each tiny dot is a single moment in time. The color of the dot represents the specific movie frame shown at that exact moment, which you can read using the color bar on the far right (ranging from frame 0 to 900).

This visualization shows exactly how each model organized the brain data. The PCA plot on the left is a disorganized, overlapping cloud, meaning it completely failed to group the data by the movie frames. The CEBRA-Time plot on the right forms a rounded shape, but the colors are still slightly mixed. The CEBRA-Behavior plot in the middle is the most successful. It forms a smooth ring where the colors progress perfectly in order. This visually proves that CEBRA-Behavior accurately mapped the continuous, complex geometry of the mouse's visual experience.

Disclaimer: We used AI to help in generate the 3D visualizations for Figure 4. The AI was used to help format the plotting code. All other data processing, model training, and conclusions are our own original work.

Figure 5: Movie Frame Decoding Accuracy

Plot: Bar chart.

Axes: The horizontal x axis lists the three different methods we tested: PCA, CEBRA-Time, and CEBRA-Behavior. The vertical y axis measures the Decoding R-squared score. This score represents how accurately our decoder could guess the correct movie frame using only the compressed 3D brain data from each model.

This chart provides the hard numbers to back up the visual evidence in the 3D scatter plots. PCA performed terribly, with a score of only 0.048. CEBRA-Time performed significantly better at 0.418. CEBRA-Behavior was the clear winner, achieving the highest accuracy at 0.571. This confirms our main hypothesis that CEBRA's behavior-guided learning is far superior to standard PCA for extracting meaningful, stimulus-related information from the primary visual cortex.

Conclusion & Discussion

Results

Our results support the idea that CEBRA's behavior-guided learning captures brain activity patterns related to the visual stimulus much better than PCA. CEBRA-Behavior achieved the highest accuracy for decoding movie frames. CEBRA-Time came in second, and PCA performed the worst. When we visualized the 3D data colored by movie frames, CEBRA-Behavior created smooth, organized paths. The PCA data however, looked scattered. This visual difference supports the idea that CEBRA successfully maps the complex paths of brain activity across the 900 movie frames. PCA just finds the largest data variations, which might not relate to the movie. Since CEBRA-Time also beat PCA, we know that simply tracking time is helpful, but adding exact frame labels gives CEBRA-Behavior a major advantage.

Project Alignment

These findings match our original project predictions. PCA captures general variance, CEBRA better separates different visual states, and the behavior-guided method wins in accuracy. This also aligns with the original CEBRA paper, showing that its benefits apply to the visual system just as they do to other brain regions.

Limitations

First, we only looked at one recording session from a single animal. This means we do not know how much the results might vary between different mice or sessions. Second, we restricted both models to three dimensions. This hurts PCA more than CEBRA because three dimensions capture only a small fraction of the total variance for PCA,

while CEBRA directly optimizes its 3D shape. Third, training CEBRA on frame numbers and then testing it on frame prediction creates a slightly circular loop. A stronger test would use completely unseen movie clips or check consistency across multiple sessions. Finally, our Ridge regression decoder is a linear model. This means our accuracy scores only measure the linear information available in the data, not the total information.

Future Directions

A logical next step is to run this analysis on multiple sessions to check consistency and measure how well CEBRA performs across different recordings. We also want to test other visual brain areas. This will help us see if CEBRA's advantage grows in brain regions that process more complex visual information. Finally, instead of using frame numbers as labels, we could use specific visual features like spatial frequency or orientation. This would test if CEBRA can capture the actual visual computations happening in the brain rather than just tracking time stamps in a movie.